

Dockerfiles & Compose Explained

Every line of this project's Docker files, what it does and why

DockerBackend/Dockerfile · DockerFe/Dockerfile · .dockerignore ·
docker-compose.yml · .env

1. Five ideas you need first

Idea	Meaning
Image	A read-only package: a small OS, a runtime (Java/Node) and your app. Built from a Dockerfile. Like a class .
Container	A running instance of an image, with its own filesystem, processes and network. Like an object . Delete it and anything written inside it is gone.
Layer	Every Dockerfile instruction creates a layer. Docker caches layers and reuses them if nothing they depend on changed. Once a layer changes, every layer after it is rebuilt.
Build context	The folder you pass to <code>docker build</code> (e.g. <code>./DockerBackend</code>). Docker sends it to the builder. <code>COPY</code> can only copy files from it.
Stage	One <code>FROM ...</code> section of a Dockerfile. In a multi-stage build only the last stage becomes the final image. Earlier stages are just build helpers.

Dockerfile instructions used in this project

Instruction	What it does
<code>FROM image AS name</code>	Start a new stage from a base image. <code>AS name</code> lets later stages refer to it.
<code>WORKDIR /app</code>	Create the folder if needed and make it the current directory for later instructions.
<code>COPY src dest</code>	Copy files from the build context into the image.
<code>COPY --from=stage</code>	Copy files out of an earlier stage instead of the build context.
<code>RUN command</code>	Run a command while building (install, compile...). Its result is saved as a layer.
<code>ARG NAME=default</code>	A variable that exists only during the build . Set it with <code>--build-arg</code> .
<code>ENV NAME=value</code>	An environment variable for later build steps and for the running container.
<code>USER name</code>	Run all later instructions and the container itself as this user, not root.
<code>EXPOSE 8080</code>	Documentation only: the app listens on 8080. It does not open the port. <code>-p / ports :</code> does.
<code>ENTRYPOINT [..]</code>	The command that runs when the container starts. Extra <code>docker run</code> arguments are appended to it.
<code>CMD [..]</code>	The default command when the container starts. Replaced if you give <code>docker run</code> a command.

Note: Both `ENTRYPOINT` and `CMD` use the **exec form** (a JSON array). The app then runs as process #1 and receives the stop signal from `docker stop` / `Ctrl+C`, so it can shut down cleanly.

2. DockerBackend/Dockerfile (Spring Boot)

A **two-stage** build. Stage 1 has Maven and a full JDK to *compile* the app. Stage 2 has only a Java runtime to *run* it. The final image contains just the JRE and one .jar file. Maven, the JDK, the source code and the downloaded dependencies all stay behind in stage 1.

```
FROM maven:3.9-eclipse-temurin-21 AS build          # ---- stage 1: build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B package -DskipTests

FROM eclipse-temurin:21-jre                          # ---- stage 2: run
WORKDIR /app
RUN groupadd --system spring && useradd --system --gid spring spring
COPY --from=build /app/target/*.jar app.jar
USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Stage 1: build the jar

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
```

Instruction	What it does and why
FROM maven:3.9-eclipse-temurin-21	Official image with Maven 3.9 and JDK 21 already installed, so nothing has to be installed by hand.
AS build	Names this stage "build" so stage 2 can say COPY --from=build.
WORKDIR /app	All the following commands run inside /app.

```
COPY pom.xml .
RUN mvn -B dependency:go-offline
```

Instruction	What it does and why
COPY pom.xml .	Copies only the dependency list first. This is the key to fast rebuilds.
RUN mvn -B dependency:go-offline	Downloads every dependency (the slow part, often minutes). This layer is cached as long as pom.xml doesn't change. -B = batch mode, quieter logs.

```
COPY src ./src
RUN mvn -B package -DskipTests
```

Instruction	What it does and why
COPY src ./src	Now copy the Java source. Editing a .java file only invalidates the cache from this line down.
RUN mvn -B package	Compiles the code and builds target/*.jar, a runnable "fat jar" with everything inside.
-DskipTests	Tests would need a running MySQL, and there is none during a build.

Stage 2: run the jar

```
FROM eclipse-temurin:21-jre
WORKDIR /app
RUN groupadd --system spring && useradd --system --gid spring spring
```

Instruction	What it does and why
FROM eclipse-temurin:21-jre	A new, clean image with only the Java Runtime : no compiler, no Maven. Smaller and with less attack surface.
RUN groupadd ... useradd ...	Creates a system user and group called spring. We'll use it so the app doesn't run as root.

```
COPY --from=build /app/target/*.jar app.jar
USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Instruction	What it does and why
COPY --from=build ...	Takes only the jar out of stage 1 and names it app.jar.
USER spring	Everything from here on, including the running app, runs as the non-root user. If the app is ever hacked, the attacker isn't root inside the container.
EXPOSE 8080	Says that Spring Boot listens on 8080. It's documentation. To publish the port you still need -p 8080:8080.
ENTRYPOINT [...]	Starts the app with java -jar app.jar when the container starts.

Layer caching in action: what reruns when you change something?

You changed...	Cached (fast)	Re-runs
A .java file	FROM, WORKDIR, COPY pom.xml, dependency:go-offline	COPY src, mvn package, stage 2 COPY
pom.xml	FROM, WORKDIR	Everything from COPY pom.xml down (all dependencies download again)
Nothing	Everything	Nothing. The build finishes in about a second.

DockerBackend/.dockerignore

Before building, Docker uploads the build context. .dockerignore lists what **not** to send:

Ignored	Why
target/	Your local build output. The image compiles its own jar, and a stale local jar could sneak in.
.git, .idea, .vscode, *.iml, .DS_Store	Version control and editor files are useless in the image and make the upload bigger.
.mvn, mvnw, mvnw.cmd	The Maven wrapper isn't needed, because the build image already has Maven.
.env	Secrets never go into an image. Anyone with the image could read them.

Ignored	Why
*.md, Dockerfile, .dockerignore	Not needed to compile or run the app.

3. DockerFe/Dockerfile (Next.js)

A **three-stage** build, all based on `node:20-alpine`. Alpine is a tiny Linux distribution of about 5 MB, which keeps the images small.

```
FROM node:20-alpine AS deps                                # ---- stage 1: install packages
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:20-alpine AS builder                             # ---- stage 2: build the app
WORKDIR /app
ARG BACKEND_URL=http://backend:8080
ENV BACKEND_URL=$BACKEND_URL
ENV NEXT_TELEMETRY_DISABLED=1
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:20-alpine AS runner                             # ---- stage 3: final image
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1
ENV PORT=3000
ENV HOSTNAME=0.0.0.0
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static
COPY --from=builder --chown=nextjs:nodejs /app/public ./public
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

Stage 1: deps (install node_modules)

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
```

Instruction	What it does and why
FROM node:20-alpine AS deps	Node.js 20 on Alpine Linux. The stage is named "deps".
COPY package.json package-lock.json ./	Copies only the package lists. Same caching trick as <code>pom.xml</code> : editing a <code>.tsx</code> file won't reinstall every package.
RUN npm ci	Installs exactly the versions in <code>package-lock.json</code> , so every build is reproducible. (<code>npm install</code> may update the lock file. <code>npm ci</code> never does.)

Stage 2: builder (next build)

```
ARG BACKEND_URL=http://backend:8080
ENV BACKEND_URL=$BACKEND_URL
ENV NEXT_TELEMETRY_DISABLED=1
```

Instruction	What it does and why
ARG BACKEND_URL=...	A build-time variable with a default. Override it with <code>docker build --build-arg BACKEND_URL=...</code> or <code>build.args</code> in <code>compose</code> .
ENV BACKEND_URL=\$BACKEND_URL	Turns the ARG into an environment variable so that <code>next.config.ts</code> can read it during <code>npm run build</code> .
ENV NEXT_TELEMETRY_DISABLED=1	Stops Next.js from sending anonymous usage data during the build.

Watch out: Why is BACKEND_URL needed at BUILD time? `next.config.ts` defines the rewrite `/api/* -> ${BACKEND_URL}/api/*`. Next.js calculates rewrites during `next build` and saves them into the build output. Changing `BACKEND_URL` only when the container *starts* has no effect. You have to rebuild the image.

```
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build
```

Instruction	What it does and why
COPY --from=deps ...	Reuses the <code>node_modules</code> installed in stage 1.
COPY . .	Copies the source code (minus everything in <code>.dockerignore</code>).
RUN npm run build	Runs <code>next build</code> . Because <code>next.config.ts</code> has output: <code>"standalone"</code> , it creates <code>.next/standalone</code> : a small <code>server.js</code> plus only the <code>node_modules</code> files it really needs.

Stage 3: runner (the final image)

```
ENV NODE_ENV=production
ENV PORT=3000
ENV HOSTNAME=0.0.0.0
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
```

Instruction	What it does and why
ENV NODE_ENV=production	Tells Node/React to run in fast production mode.
ENV PORT=3000	<code>server.js</code> listens on this port.
ENV HOSTNAME=0.0.0.0	Listen on all network interfaces. With <code>localhost</code> the server would only accept connections from inside the container itself, and port publishing would not work.
RUN addgroup ... adduser ...	Creates a non-root user <code>nextjs</code> . Alpine uses <code>addgroup/adduser</code> instead of <code>groupadd/useradd</code> .

```

COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static
COPY --from=builder --chown=nextjs:nodejs /app/public ./public
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]

```

Instruction	What it does and why
.next/standalone	The minimal server and its dependencies. No full node_modules and no source code.
.next/static	The JS/CSS bundles that the browser downloads. Standalone output doesn't include them, so we copy them ourselves.
public	Static files such as images and icons.
--chown=nextjs:nodejs	Makes the non-root user the owner of the copied files.
USER nextjs	Run the server as the non-root user.
CMD ["node", "server.js"]	Starts the Next.js production server.

Dockerfile.dockerignore

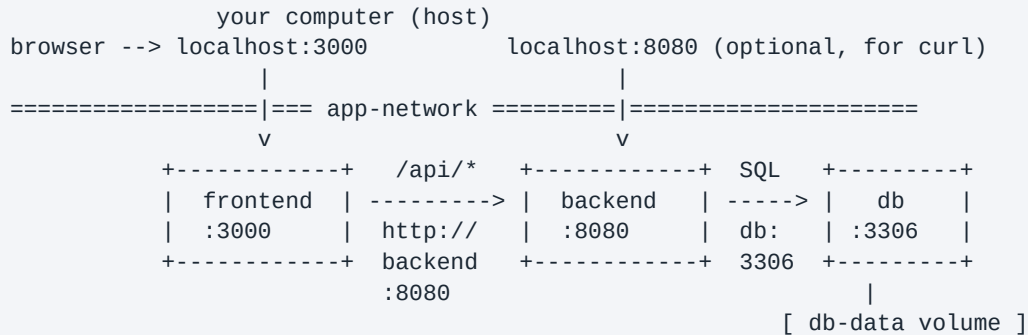
Ignored	Why
node_modules	Can be hundreds of MB, which makes the upload slow. It may also contain Mac/Windows binaries that break on Linux. The image runs its own npm ci.
.next, out	Your local build output. The image builds its own.
.git, .idea, .vscode, .DS_Store	Not needed in the image.
.env*	Secrets must never be baked into an image.
npm-debug.log*, *.md, Dockerfile, .dockerignore	Logs and docs aren't needed to build or run the app.

Image size: why multi-stage matters

Image	Contains	Size (approx.)
Backend build stage	Maven + full JDK + sources + all dependencies	much larger (discarded)
Backend final image	Ubuntu + JRE 21 + app.jar	~580 MB
Frontend final image	Alpine + Node 20 + standalone server	~290 MB

4. docker-compose.yml

Compose describes the **whole application** (all containers, plus their network and storage) in one file. `docker compose up --build` replaces a whole list of manual `docker network create`, `docker build` and `docker run` commands.



The file's structure

```
services:      # the containers: db, backend, frontend
  db: ...
  backend: ...
  frontend: ...
networks:      # the private network they share
  app-network: ...
volumes:       # persistent storage managed by Docker
  db-data:
```

Variables: \${VAR:-default} and \$\$

Syntax	Meaning
\${DB_NAME}	Replaced with the value from the <code>.env</code> file (or your shell environment).
\${DB_NAME:-messages_db}	Use <code>DB_NAME</code> if it is set, otherwise <code>messages_db</code> . So compose works even without a <code>.env</code> file.
\$\$MYSQL_ROOT_PASSWORD	<code>\$\$</code> is an escaped <code>\$</code> . Compose leaves it alone, so the variable is expanded later inside the container .

Service: db (MySQL)

```
db:
  image: mysql:8.4
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root_password}
    MYSQL_DATABASE: ${DB_NAME:-messages_db}
    MYSQL_USER: ${DB_USER:-app}
    MYSQL_PASSWORD: ${DB_PASSWORD:-app_password}
  volumes:
    - db-data:/var/lib/mysql
```

Instruction	What it does and why
image: mysql:8.4	Use the official image from Docker Hub as it is. No Dockerfile needed.
environment:	The MySQL image reads these on its first start (empty volume) to set the root password and create a database plus a normal user. Changing them later does nothing until you run <code>down -v</code> .
volumes: db-data:/var/lib/mysql	Mounts the named volume db-data at MySQL's data folder. Data is stored outside the container, so it survives <code>docker compose down</code> .
(no ports:)	The db is not published to your computer. The backend reaches it over the internal network, which needs no publishing. Add <code>ports: ["3306:3306"]</code> if you want to connect with a desktop SQL tool.

```
healthcheck:
  test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$MYSQL_ROOT_PASSWORD --silent"]
  interval: 5s
  timeout: 5s
  retries: 20
  start_period: 20s
  restart: unless-stopped
  networks:
    - app-network
```

Instruction	What it does and why
healthcheck.test	A command Docker runs again and again inside the container. Exit code 0 means healthy . <code>CMD-SHELL</code> runs it through a shell, so <code>\$MYSQL_ROOT_PASSWORD</code> is expanded.
-h 127.0.0.1	Forces a TCP connection. During its first-time setup MySQL only listens on a local socket, so the check only passes once MySQL is really ready.
interval / timeout	Check every 5 s. A check that takes longer than 5 s counts as failed.
retries: 20	After 20 failures in a row, the container is marked unhealthy .
start_period: 20s	A grace period at startup: failures during this time don't count.
restart: unless-stopped	Restart the container if it crashes or Docker restarts, unless you stopped it.
networks: app-network	Join the shared private network.

Service: backend (Spring Boot)

```
backend:
  build:
    context: ./DockerBackend
  environment:
    DB_HOST: db
    DB_PORT: 3306
    DB_NAME: ${DB_NAME:-messages_db}
    DB_USER: ${DB_USER:-app}
    DB_PASSWORD: ${DB_PASSWORD:-app_password}
  depends_on:
    db:
      condition: service_healthy
```

Instruction	What it does and why
build.context	Build the image from DockerBackend/Dockerfile, the same as docker build ./DockerBackend.
DB_HOST: db	Service discovery: Docker's built-in DNS turns the service name db into the db container's IP address. Never use localhost here: inside a container, localhost is the container itself.
DB_NAME / DB_USER / DB_PASSWORD	Must match what MySQL created. application.yml reads them as \${DB_HOST:localhost} and so on.
depends_on ... service_healthy	Don't start the backend until db's healthcheck passes. A plain depends_on: [db] only waits for the container to <i>start</i> , and MySQL isn't ready then yet.

```
ports:
  - "8080:8080"
healthcheck:
  test: ["CMD-SHELL", "curl -fs http://localhost:8080/actuator/health || exit 1"]
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 30s
restart: unless-stopped
networks:
  - app-network
```

Instruction	What it does and why
ports: "8080:8080"	HOST:CONTAINER. Your computer's port 8080 forwards to the container's port 8080, so curl localhost:8080/... works from your machine.
curl -fs .../actuator/health	Spring Actuator reports UP only when the app and its DB connection work. -f makes curl fail on HTTP errors. Here localhost is correct, because the check runs <i>inside</i> the backend container.
start_period: 30s	Spring Boot needs a few seconds to start, so early failures are ignored.

Service: frontend (Next.js)

```
frontend:
  build:
    context: ./DockerFe
    args:
      BACKEND_URL: http://backend:8080
  environment:
    BACKEND_URL: http://backend:8080
  depends_on:
    backend:
      condition: service_healthy
  ports:
    - "3000:3000"
  restart: unless-stopped
  networks:
    - app-network
```

Instruction	What it does and why
build.args	Passed to ARG BACKEND_URL in the Dockerfile. Needed at build time, because Next.js saves the rewrite into the build output.
environment: BACKEND_URL	Also set at runtime for consistency. You can see it with <code>docker compose exec frontend env</code> .
http://backend:8080	The backend's service name and its container port. Inside the network you always use the container port, never the published host port.
depends_on backend healthy	The frontend starts only once the backend's healthcheck passes.
ports: "3000:3000"	Open <code>http://localhost:3000</code> in your browser.

Top-level networks and volumes

```
networks:
  app-network:
    driver: bridge

volumes:
  db-data:
```

Instruction	What it does and why
networks.app-network	A user-defined bridge network: a private virtual network on your machine. Services on it can reach each other by name, and outside containers can't get in. (Compose would create a default network anyway. Declaring it makes the idea visible.)
driver: bridge	The standard single-machine network type.
volumes.db-data	A named volume managed by Docker. List volumes with <code>docker volume ls</code> . Compose prefixes it with the project name, e.g. <code>rootfolder_db-data</code> .

ports vs EXPOSE vs no port at all

	Who can connect?	Used in this project
EXPOSE 8080 (Dockerfile)	Nobody extra. It's only documentation.	Both Dockerfiles
ports: "8080:8080"	Your computer (localhost:8080) and other containers	backend, frontend
nothing	Only containers on the same network (db:3306)	db

restart policies

Policy	Behaviour
no (default)	Never restart automatically.
on-failure	Restart only if the app exits with an error.
always	Always restart. Even a container you stopped yourself starts again when Docker restarts.
unless-stopped	Like always, except when you stopped it yourself. Used in this project.

5. .env and .env.example

```
MYSQL_ROOT_PASSWORD=root_password
DB_NAME=messages_db
DB_USER=app
DB_PASSWORD=app_password
```

File	Purpose
.env.example	A template that is shared and committed. It shows which variables exist.
.env	Your real values, created with <code>cp .env.example .env</code> . Compose reads it automatically from the folder containing <code>docker-compose.yml</code> . Never commit it.

6. Putting it together: one request, step by step

What happens when you click **Add** on `http://localhost:3000`:

#	Step
1	The browser (on your computer) sends <code>POST /api/messages</code> to localhost:3000 .
2	Docker forwards host port 3000 to the frontend container's port 3000 (ports: <code>"3000:3000"</code>).
3	Next.js matches the rewrite <code>/api/:path*</code> and forwards the request to <code>http://backend:8080/api/messages</code> .
4	Docker's DNS on app-network resolves <code>backend</code> to the backend container's IP.
5	Spring Boot saves the message using <code>jdbc:mysql://db:3306/messages_db</code> . Here <code>db</code> is resolved the same way.
6	MySQL writes the row into <code>/var/lib/mysql</code> , which is the db-data volume.
7	The answer travels back: <code>backend -> frontend -> browser</code> . The browser only ever talked to <code>localhost:3000</code> .

Note: This is why the browser can't call `http://backend:8080` directly. The name `backend` only exists inside the Docker network, and the browser runs outside it. The frontend server runs inside the network, so it can act as the go-between. A bonus is that the browser sees a single origin, so no CORS configuration is needed.